

dorm LINK

Campus Oriented Hostel Manager

Project Report

A full-stack web application with separate Admin and Student portals for hostel booking, room allocation, fees, complaints, announcements and QR-based entry/exit logging.

Stack: HTML / CSS / JavaScript • Java (Jakarta Servlets) • MySQL (XAMPP)

1. System Overview

dorm_LINK is a web-based hostel management system for educational campuses. It provides two role-based portals — Admin and Student — backed by a layered Java servlet API and a MySQL database. The interface follows a neumorphism + glassmorphism design language with an SF Pro typographic system and a vibrant “chameleon” hover treatment on interactive elements.

Core modules

- Authentication (login, sign-up, email-OTP password reset)
- Hostel booking and room allocation workflow
- Fee management with payments and downloadable receipts
- Complaint handling (Maintenance, Food, Electricity, Water, Others)
- Announcements (hostel, campus, university)
- QR-based entry/exit attendance logging
- Ratings and weekly mess-menu management

2. Technology Stack

Layer	Technology
Frontend	HTML5, CSS3 (neumorphism + glassmorphism), vanilla JavaScript
Backend	Java 17, Jakarta Servlets (Tomcat 10.1+)
Database	MySQL 8 via XAMPP / phpMyAdmin (JDBC)
Auth	Salted SHA-256 password hashing; in-memory email OTP
Build/Deploy	javac + WAR-style layout on Apache Tomcat

3. System Architecture

The backend follows a clean layered architecture. Each HTTP request enters a thin servlet controller, which delegates business logic to a service, which uses a DAO for JDBC data access; DAOs map rows to POJO models that serialise themselves to JSON.

Layer	Folder	Responsibility
Controllers	backend/controllers	Jakarta servlets, one per module, mapped under /api/*
Services	backend/services	Validation and business rules (e.g. approve→allocate)
DAO	backend/dao	Prepared-statement JDBC data access
Models	backend/models	POJOs with toJson() serialisation
Utils	backend/utils	DBConnection, PasswordUtil, OtpUtil, QrUtil, JsonUtil, AuthUtil
Config	backend/config	DatabaseConfig (reads db.properties)

The static frontend is served at the web-app root and calls the API at /dormlink/api/. . . . List responses are wrapped as {"data": [. . .]} and unwrapped client-side. If the backend is offline, every page degrades to a labelled preview with sample data.

4. Data Model (ER summary)

Principal entities and relationships (full DDL in database/schema.sql):

- COURSE 1—N STUDENT • HOSTEL 1—N ROOM • ROOM 1—N STUDENT (capacity controlled)
- STUDENT 1—N ROOM_REQUEST • ROOM 1—N ROOM_REQUEST • STUDENT 1—N BOOKING
- STUDENT 1—N COMPLAINT • STUDENT 1—N FEE • FEE 1—N PAYMENT
- STUDENT 1—N ENTRY_EXIT_LOG • STUDENT/HOSTEL 1—N RATING • HOSTEL 1—N MESS_MENU
- ADMIN 1—N ANNOUNCEMENT and ROOM_REQUEST approvals

Database triggers keep `hostel.avg_rating` and `room.status` consistent automatically, so hostel ratings update in real time on the Book Hostel cards.

5. Key Process Flows (DFD)

Authentication

Login request → credential validation → dashboard; Forgot password → OTP → reset.

Hostel booking

Browse hostels → request room → availability check → admin approval → allocation (`room_id` set, `occupancy++`).

Fees

View dues → pay (single transaction) → status recomputed → receipt generated.

Entry/Exit

Scan gate QR → verify roll → Log Entry / Log Exit → visible to student and admin.

6. REST API Surface

Module	Base path	Representative endpoints
Auth	/api/auth	login, register, forgot, reset
Rooms	/api/rooms	list, hostels, mess, mine, rate, requests, requests/{id}
Students	/api/students	list, courses, stats, {id}
Complaints	/api/complaints	mine, (list), POST
Fees	/api/fees	mine, dues, receipts, pay, remind/{id}
Logs	/api/logs	mine, (list), POST
Announcements	/api/announcements	list (?all), POST, {id}/toggle

7. Running the Project

- Import database/schema.sql then database/seed_data.sql in phpMyAdmin.
- Set MySQL credentials in backend/config/db.properties.
- Compile backend/** against the servlet API + MySQL connector; deploy with frontend/ and uploads/ on Tomcat 10.1+.
- Open /dormlink/frontend/html/auth/signin.html.
- Demo logins: admin/admin123, warden/warden123, USTM2024CS001/student123.

Full setup instructions are in docs/INSTALLATION_GUIDE.md. Folder layout and per-module API specs accompany the source.